

Nome do estudante: _____ Código UP: _____ Nº Prova: _____

1. [4.5]

Indique quais das seguintes afirmações são verdadeiras e quais são falsas, assinalando respetivamente com **V** ou **F**.

NOTA - A pontuação nesta pergunta será dada pela fórmula: máximo (0, (nº_respostas_certas – nº_respostas_erradas) * 0.3)

	V / F
Em programação POSIX, uma <i>condition variable</i> deve ser sempre usada em associação com um <i>mutex</i> .	
O código que implementa as primitivas wait() e signal() que operam sobre semáforos constitui uma secção crítica que pode ser implementada com recurso a instruções máquina especiais como, por exemplo, <i>test-and-set</i> .	
No contexto do escalonamento de processos, a política de "envelhecimento" tem em vista evitar a inanição de alguns processos.	
O código seguinte escreve na saída padrão o número 100 : int x=100; write(STDOUT_FILENO,&x,3);	
Uma secção crítica é um pedaço de código que tem de ser executado sem interrupções.	
É possível que dois processos acedam por endereços lógicos diferentes ao mesmo endereço físico.	
Em Linux, na sequência de uma chamada fork() , o processo pai e o processo filho partilham as variáveis globais, podendo o processo pai manipular as variáveis do processo filho e vice-versa.	
A multiprogramação só pode ser implementada em computadores com vários processadores (<i>CPUs</i>) ou com um processador multinúcleo (<i>multicore</i>).	
Quando vários processos pretendem acrescentar informação no fim de um ficheiro partilhado, convém que o ficheiro seja aberto em modo O_APPEND , caso contrário, a operação de escrita deve ser feita numa secção crítica envolvendo as chamadas lseek() e write() .	
Uma das condições necessárias para garantir a segurança do sistema operativo é o processador (<i>CPU</i>) suportar um modo de operação privilegiado.	
No contexto da gestão de memória virtual a ocorrência de uma falta de página leva à terminação imediata do processo que a originou.	
Um dos problemas da utilização de sinais como mecanismo de comunicação entre processos é que pode acontecer que nem todos os sinais enviados a um processo sejam recebidos.	
Uma das vantagens do uso de <i>threads</i> sobre o uso de processos é que não é necessário usar memória partilhada ou mecanismos de intercomunicação, como <i>pipes</i> , para a comunicação entre <i>threads</i> de um mesmo processo.	
Em Linux, as funções signal() e pthread_cond_signal() têm funcionalidades semelhantes, devendo a primeira ser usada com processos e a segunda usada com <i>threads</i> .	
As chamadas exec fazem simultaneamente duas coisas: criam um novo processo e carregam o código de um programa a executar nesse processo.	

Nome do estudante: _____ Código UP: _____ Nº Prova: _____

2. [4.5]

a) [2.0] Complete as frases seguintes com uma palavra ou um curto texto.

O mecanismo de <i>hardware</i> conhecido por DMA (<i>Direct Memory Access</i>) é fundamental para ...
Ao longo da sua existência, os processos/ <i>threads</i> vão comutando entre 3 estados principais que são: ...
A forma mais simples de prevenir <i>deadlocks</i> nos programas dos utilizadores é ...
Uma elevada atividade do disco acompanhada de uma baixa utilização do processador pode ser um sintoma de ...
Em Linux, a estrutura de dados que descreve ficheiros e diretórios armazenados em disco é designada por ...

b) [1.5] Três *threads*, **T1**, **T2** e **T3**, de um mesmo programa, têm três blocos de código, respetivamente, **f1()**, **f2()** e **f3()** que se pretende que sejam executados sequencial e repetidamente, começando sempre por **f1()**, seguindo-se **f2()** e **f3()**, por qualquer ordem, mas com a restrição que **f2()** e **f3()** nunca devem ser executados simultaneamente; **f1()** só será executado novamente após **f2()** e **f3()** terem concluído a sua execução; isto é, a sequência de execução deverá ser: **f1()** → [**f2()**→**f3()** ou **f3()**→**f2()**] e novamente **f1()** → [**f2()**→**f3()** ou **f3()**→**f2()**] -

<u>T1</u>	<u>T2</u>	<u>T3</u>
...	...	...
while(1) {	while(1) {	while(1) {
...	...	...
f1();	f2();	f3();
...	...	...
}	}	}
...	...	...

inicialização dos semáforos	T1	T2	T3
-----------------------------	----	----	----

c) [1.0] Num sistema que usa paginação a pedido (*demand paging*) qual a importância de cada página ter associado um bit de referência (*reference bit*) e um bit de modificação (*modified bit*)?

Nome do estudante: _____ Código UP: _____ Nº Prova: _____

Nota: nesta prova apenas é necessário fazer tratamento de erros e indicar os ficheiros de inclusão quando tal for solicitado explicitamente

3. [4.5]

O programa **removebig** remove de um diretório todos os ficheiros regulares cujo tamanho seja superior a um dado valor. Os argumentos do programa são o nome do diretório e o referido tamanho (exemplo de invocação para remover os ficheiros do diretório **dir1** cujo tamanho seja superior a 1000000: **removebig dir1 1000000**). **Nota: os subdiretórios devem ser ignorados.**

a) [2.0] Escreva a parte do código de **removebig** que percorre o referido diretório e, por cada ficheiro regular encontrado cujo tamanho seja superior ao indicado, escreve o nome do ficheiro na saída padrão, incrementa um contador do número de ficheiros a remover (variável **count**) e invoca a função **void remove(char *filename)**, cujo parâmetro é o nome do ficheiro a remover. Não implemente por enquanto esta função.

b) [1.5] Escreva o código da função **remove()** que remove o ficheiro cujo nome recebe como parâmetro, recorrendo para isso a um subprocesso (processo filho) que executa o utilitário **rm**. **Nota: não use chamadas `system()`.**

c) [1.0] Escreva a parte do código de **removebig** que espera que todos os processos filhos terminem e escreve na saída padrão o número total de ficheiros removidos.

4. [4.0]

Considere dois programas (**server** e **client**). O processo **server** recebe, através de um *pipe* com nome (FIFO), "mensagens" que lhe são enviadas, direta ou indiretamente, por um ou mais processos **client**; as mensagens são *strings* de tamanho fixo; o processo **server** termina quando receber uma mensagem vazia (""). Um processo **client** recebe como argumento da linha de comando o nome de um ficheiro e recorre a um programa auxiliar (uma versão personalizada do programa **md5sum**) para calcular o "resumo criptográfico" do ficheiro (um número hexadecimal). O programa **md5sum** recebe como argumento o nome de um ficheiro e apresenta na saída padrão o referido número hexadecimal (ver abaixo um exemplo de execução de **md5sum**, na linha de comandos). Pretende-se enviar a saída do programa **md5sum** para o processo **server**. O programa **server** mostra essa saída (ou "mensagem") no ecrã e faz um processamento que não importa especificar, neste contexto.

Foram desenvolvidas duas implementações alternativas para o programa **client** que se pretende comparar e avaliar.

Exemplos de execução	Programa client
Programa client: \$./client image.png \$./client document.pdf \$./client "" <i>//NOTA: envia mensagem vazia ao servidor</i> Programa server: \$./server dcf92388dd338dbb74ad1b08e61091a7 7b7a53e239400a13bd6be6c91c4f6c4e <i>//NOTA: servidor foi encerrado (foi recebida uma mensagem vazia)</i> Programa md5sum: \$ md5sum image.png dcf92388dd338dbb74ad1b08e61091a7	<pre>// #include ... #define MAX_SIZE 32 #define CHANNEL "/tmp/server" int main(int argc, char *argv[]) { char cmd[256], msg[MAX_SIZE] = ""; int fd; while ((fd = open(CHANNEL, O_WRONLY)) < 0) ; if (argv[1][0]!='\0') { write(fd, msg, MAX_SIZE); close(fd); return 0; } ... // código da implementação (ver alternativas abaixo) }</pre>
Implementação 1	Implementação 2
<pre>snprintf(cmd, sizeof(cmd), "md5sum %s", argv[1]); FILE *f = popen(cmd, "r"); fread(msg, MAX_SIZE, 1, f); write(fd, msg, MAX_SIZE); close(fd); fclose(f); return 0;</pre>	<pre>dup2(fd, STDOUT_FILENO); close(fd); execlp("md5sum", "md5sum", argv[1], NULL); exit(1);</pre>

a) [1.0] Explique sucintamente a diferença entre as duas implementações; recorra a figura(s), se achar conveniente.

Implementação 1	Implementação 2

b) [1.5] Indique, se existir e justificando sempre com uma frase, qual a implementação mais indicada nos seguintes casos: **(1)** **md5sum** escreve na saída carácter a carácter, **(2)** **md5sum** escreve na saída de uma só vez, **(3)** **MAX_SIZE** passa a ser superior a **PIPE_BUF**.

(1) (2) (3)

c) [1.5] Complete o código do programa **server**.

<pre>int main() { char msg[MAX_SIZE + 1] = { '\0' }; // todas os elementos serão preenchidos com '\0' int fd, nr; mkfifo(,); fd = open(,); while (((nr = read(, ,)) > 0) && (msg[0] != '\0')) { printf("%s\n",); // ... escrita da mensagem recebida // ... processamento da mensagem recebida } unlink(); return 0; }</pre>
--

Nome do estudante: _____ Código UP: _____ Nº Prova: _____

5. [2.5]

Recorde o problema **2.b** desta prova. Considerando uma implementação baseada em semáforos e/ou mutexes POSIX, responda às questões seguintes.

a) [0.9] Escreva o cabecalho da função **thread2()**, que implementa o *thread* designado por **T2** no problema **2.b**, e as partes do seu código referentes à sincronização da execução com os restantes *threads*. Considere que o *thread* não recebe parâmetros.

b) [0.8] Tendo em conta que os *threads* pertencem ao mesmo programa, diga onde deve(m) ser definido(s) o(s) semáforo(s) e/ou mutex(es) e onde e quando deve ser feita a sua inicialização. Dê um exemplo de declaração e inicialização de cada tipo de mecanismo de sincronização que selecionou.

c) [0.8] Se, em vez de pertencerem ao mesmo programa, os *threads* **T1**, **T2** e **T3** pertencessem a programas diferentes, que alterações seria necessário introduzir na definição e inicialização dos mecanismos de sincronização utilizados? Não escreva código.

FIM